

INFORME DE PROYECTO

DISEÑO E IMPLEMENTACIÓN DE UNA BASE DE DATOS ORACLE Y PL/SQL PARA UN SISTEMA DE GESTIÓN DE HOTEL Y RESERVAS

Asignatura: Taller de Base de Datos
Estudiantes: Boris Astudillo — Dennis Ponce
Docente: Humberto Geovanni Marinao Vidal

Fecha: 16 de Septiembre del 2026

Tabla de Contenidos

1. Introducción
2. Contexto y Requerimientos del Negocio
3. Datos e Información del Sistema
4. Diseño de la Base de Datos
5. Tipos de Datos Compuestos
6. Procesamiento mediante Cursores
7. Manejo de Excepciones
8. Procedimientos y Funciones Almacenadas (Recomendaciones)
9. Packages (Recomendación)
10. Triggers (Recomendación)
11. Consideraciones Técnicas
12. Conclusiones
13. Anexos

1. Introducción

1.1 Descripción del proyecto

Este proyecto desarrolla la capa de base de datos de un Sistema de Gestión de Hotel y Reservas. La solución centraliza la información necesaria para administrar huéspedes, empleados, habitaciones, tipos de habitación, tarifas, reservas, estadías, servicios adicionales, consumos, cuentas de cobro y pagos. La base de datos permite mantener los datos organizados y relacionados para apoyar las operaciones del hotel.

Oracle Database es el motor utilizado. SQL se emplea para definir la estructura, relaciones, restricciones y datos iniciales, mientras que PL/SQL incorpora procesamiento procedural. La primera evaluación se concentra en bloques anónimos, RECORD, VARRAY, cursores explícitos, cursores parametrizados, loops, loops anidados, cursores complejos y excepciones.

1.2 Problemática

El problema principal es mantener información centralizada, organizada y consistente cuando existen múltiples entidades relacionadas. Una reserva debe asociarse a un huésped, a una o más habitaciones y a fechas determinadas; posteriormente puede relacionarse con una estadía, ocupantes, consumos, una cuenta y pagos.

Un problema concreto es determinar la disponibilidad de una habitación para determinadas fechas. No basta con revisar el estado actual: también es necesario comprobar que no exista una reserva que se superponga con el período solicitado.

1.3 Objetivo general

Diseñar e implementar una solución de base de datos Oracle para gestionar la información hotelera, manteniendo sus relaciones e integridad y utilizando PL/SQL para procesar, consultar y controlar información relevante del negocio.

1.4 Objetivos específicos y alcance

- Representar las entidades y relaciones principales mediante un modelo relacional.
- Mantener la integridad mediante claves y restricciones.
- Aplicar RECORD y VARRAY para organizar información en PL/SQL.
- Procesar conjuntos de filas mediante cursores con y sin parámetros.
- Aplicar loops y loops anidados para recorrer información relacionada.
- Resolver la consulta de disponibilidad mediante un cursor complejo.
- Controlar situaciones de error mediante excepciones predefinidas y propias.

El desarrollo principal de la primera evaluación llega hasta excepciones. Procedimientos, funciones, packages y triggers se mantienen como recomendaciones de evolución para la

segunda evaluación. Las aplicaciones web, móviles y backend de integración quedan fuera de esta etapa.

2. Contexto y Requerimientos del Negocio

2.1 Descripción del negocio

El hotel administra habitaciones de distintos tipos, tarifas asociadas a temporadas, huéspedes y reservas. Una reserva puede incluir una o varias habitaciones y cada detalle registra sus fechas. La reserva puede dar origen a una estadía, durante la cual se registran ocupantes y consumos de servicios. Estos consumos pueden asociarse a una cuenta de cobro y posteriormente a pagos.

Flujo principal: HUÉSPED → RESERVA → HABITACIÓN → ESTADÍA → SERVICIOS /
CONSUMOS → CUENTA → PAGO

2.2 Actores del sistema

Actor	Rol
Recepcionista	Registra huéspedes, crea reservas, realiza check-in y registra servicios y pagos.
Encargado(a) de reservas	Gestiona, confirma o cancela reservas.
Administrador(a) / Supervisor	Consulta información consolidada y gestiona tarifas y catálogos.

2.3 Requerimientos funcionales (RF)

Código	Requerimiento
RF-01/02	Cada habitación pertenece a un tipo y no debe existir una asignación con fechas superpuestas.
RF-03/04	Una reserva puede incluir varias habitaciones, cada una con fecha de ingreso y salida.
RF-05/06	El valor de la estadía depende del tipo de habitación y las noches; los servicios se cargan a la estadía.
RF-07/08	La reserva transita entre Pendiente, Confirmada, Cancelada y Finalizada; el check-in exige una reserva Confirmada.
RF-09/10	Al check-out se determina el saldo pendiente y se permite consultar disponibilidad por período.

Estos requerimientos permiten conectar el problema del negocio con los recursos PL/SQL. El caso más relevante para la primera evaluación es RF-10, consulta de disponibilidad por período.

3. Datos e Información del Sistema

3.1 Datos almacenados

La base de datos contempla 22 tablas organizadas en grupos funcionales:

Grupo	Entidades
Catálogos	pais, ciudad, tipo_documento, estado_habitacion, estado_reserva, categoria_servicio, metodo_pago, temporada
Huéspedes / habitaciones	huesped, empleado, tipo_habitacion, habitacion, tarifa_habitacion
Reservas / estadías	reserva, detalle_reserva, auditoria_reserva, estadía, ocupante_estadía
Servicios	servicio_adicional, consumo_servicio
Cuentas / pagos	cuenta_cobro, pago

El script considera 130 registros iniciales y 13 secuencias para apoyar la generación de identificadores. Esto proporciona un escenario de datos suficiente para ejecutar y comprobar los ejercicios PL/SQL.

3.2 Información generada por el sistema

- Habitaciones disponibles para un rango de fechas.
- Información agrupada de huéspedes y reservas.
- Información relacionada de reservas, habitaciones y estadías.
- Datos de servicios y consumos.
- Información de cuentas de cobro y pagos.
- Mensajes de control ante condiciones de error.

El dato corresponde al valor almacenado; la información surge cuando esos datos se consultan, relacionan o procesan para responder una necesidad del negocio.

4. Diseño de la Base de Datos

4.1 Modelo de datos

El modelo relacional representa las entidades mediante tablas relacionadas por claves primarias y foráneas. Esto permite mantener la trazabilidad entre las operaciones del hotel y sirve como base para las consultas y el procesamiento PL/SQL.

A continuación se incorporan los diagramas reales utilizados en el proyecto.



Las relaciones principales permiten representar el flujo huésped–reserva–detalle de reserva–habitación y, posteriormente, estadía, ocupantes, consumos, cuenta y pago.

5. Tipos de Datos Compuestos

5.1 RECORD

RECORD es un tipo de dato compuesto que agrupa varios campos relacionados dentro de una misma estructura. En el proyecto permite manejar como una unidad lógica información de entidades como huéspedes o reservas.

```
TYPE t_huesped IS RECORD (
    id          huesped.id_huesped%TYPE,
    nombre      huesped.nom_huesped%TYPE,
    apellido    huesped.ape_huesped%TYPE,
    documento   huesped.num_documento%TYPE
);
```

```
v_huesped t_huesped;
```

La consulta obtiene los datos y RECORD permite almacenarlos y manipularlos agrupadamente. %TYPE mantiene el tipo alineado con el campo de la tabla.

5.2 VARRAY

VARRAY es una colección ordenada cuyo tamaño máximo se define al declararla. Es apropiado para trabajar temporalmente con una cantidad limitada de elementos relacionados.

```
TYPE t_nombres IS VARRAY(5) OF VARCHAR2(100);
v_nombres t_nombres := t_nombres();

v_nombres.EXTEND;
v_nombres(1) := 'Dato obtenido desde la consulta';
```

5.3 Justificación

Estructura	Uso	Razón
RECORD	Agrupar campos de una entidad.	Trata atributos relacionados como una unidad lógica.
VARRAY	Colección limitada y ordenada.	Adecuado cuando la cantidad se encuentra acotada.

La elección depende del problema: RECORD representa un registro compuesto; VARRAY representa varios elementos de una colección.

6. Procesamiento mediante Cursores

6.1 Cursores explícitos sin y con parámetros

Un cursor explícito permite recorrer un conjunto de filas. Los ejercicios sin parámetros recorren consultas definidas previamente; los parametrizados reciben valores de entrada y permiten reutilizar la misma consulta para distintas condiciones.

6.2 Cursor complejo de disponibilidad

El caso más relevante es `c_disponibilidad`, que recibe las fechas solicitadas y relaciona habitaciones, tipos, estados y reservas para determinar qué habitaciones pueden ser ofrecidas.

```
CURSOR c_disponibilidad(
    p_ingreso DATE,
    p_salida DATE
) IS
    SELECT h.id_habitacion,
           h.numero_hab,
           t.nombre_tipo,
           h.piso
    FROM habitacion h
    JOIN tipo_habitacion t
      ON t.id_tipo_hab = h.id_tipo_hab
    JOIN estado_habitacion eh
      ON eh.id_estado_hab = h.id_estado_hab
    WHERE eh.nombre_estado = 'Disponible'
    AND NOT EXISTS (
        SELECT 1
        FROM detalle_reserva d
        JOIN reserva r
          ON r.id_reserva = d.id_reserva
        WHERE d.id_habitacion = h.id_habitacion
              AND d.fec_ingreso < p_salida
              AND d.fec_salida > p_ingreso
    );
```

Los parámetros representan el período solicitado. NOT EXISTS excluye una habitación cuando existe una reserva cuyo intervalo se cruza con el período consultado. La condición de solapamiento es `d.fec_ingreso < p_salida` y `d.fec_salida > p_ingreso`.

6.3 Loops anidados

Los loops anidados permiten recorrer dos niveles relacionados: por cada registro principal se procesan registros asociados. En el proyecto se aplican a relaciones uno a muchos, por ejemplo tipo–habitación o estadía–consumo.

6.4 Justificación

Los cursores se utilizan cuando se necesita recorrer múltiples filas y aplicar procesamiento procedural. Los parametrizados agregan reutilización y los complejos permiten combinar relaciones y condiciones del negocio. En grandes volúmenes, el procesamiento fila a fila debe evaluarse frente a alternativas orientadas a conjuntos.

7. Manejo de Excepciones

7.1 Predefinidas de Oracle

Las excepciones predefinidas permiten controlar situaciones conocidas durante la ejecución. El proyecto utiliza especialmente `NO_DATA_FOUND` cuando un `SELECT INTO` no encuentra el registro esperado.

Excepción	Situación
NO_DATA_FOUND	SELECT INTO no encuentra ninguna fila.
TOO_MANY_ROWS	SELECT INTO devuelve más de una fila cuando se esperaba una.
DUP_VAL_ON_INDEX	Se intenta ingresar un valor que viola una restricción única.
ZERO_DIVIDE	Se intenta dividir por cero.

7.2 Excepciones definidas por el usuario

Las excepciones propias permiten representar reglas específicas del negocio, como controlar un pago que supera el monto permitido o el intento de realizar check-in sobre una reserva cancelada.

```
e_pago_excesivo EXCEPTION;

IF p_monto > v_total THEN
    RAISE e_pago_excesivo;
END IF;
```

7.3 Estrategia e integridad

Las excepciones se capturan en la sección EXCEPTION del bloque PL/SQL para controlar situaciones previstas y entregar mensajes comprensibles. Las restricciones protegen la estructura de los datos y las excepciones complementan ese control durante el procesamiento.

En el contexto del hotel, esto permite detener o informar una operación cuando una condición necesaria no se cumple, en lugar de continuar como si el resultado fuera válido.

8. Procedimientos y Funciones Almacenadas (Recomendaciones)

Esta sección corresponde a una evolución para la segunda evaluación y no al foco principal de la primera.

8.1 Procedimientos

Se recomienda encapsular operaciones reutilizables como registrar una reserva, registrar un pago o centralizar una operación de disponibilidad.

8.2 Funciones

Se recomienda utilizar funciones para cálculos que deban retornar un valor, por ejemplo cantidad de noches o saldo pendiente. La elección se justifica cuando el resultado principal de la operación es un valor reutilizable.

9. Packages (Recomendación)

En una etapa posterior, las operaciones relacionadas con reservas podrían organizarse en un package, reuniendo procedimientos y funciones en una unidad lógica para favorecer organización, reutilización y mantenimiento.

10. Triggers (Recomendación)

Como evolución futura, los triggers pueden utilizarse cuando una acción deba ejecutarse automáticamente ante un INSERT, UPDATE o DELETE. En el sistema hotelero podrían apoyar auditoría o validaciones de eventos. Deben utilizarse con criterio porque agregan lógica implícita a las operaciones.

Los puntos 8, 9 y 10 se mantienen como recomendaciones para la segunda evaluación, no como contenido principal de la defensa de esta etapa.

11. Consideraciones Técnicas

11.1 Rendimiento

El rendimiento depende del volumen de datos, costo de las consultas, filas procesadas y concurrencia. Los cursores y loops deben utilizarse cuando existe una necesidad procedural clara.

Como planificación futura pueden probarse escenarios de 50, 100, 250 y 500 transacciones por minuto. Son escenarios de prueba y no capacidades garantizadas; las pruebas reales deberían medir tiempos de respuesta y uso de recursos.

11.2 Mantenibilidad

Una estructura separada por tablas, restricciones y lógica PL/SQL facilita identificar responsabilidades. En una etapa posterior, procedimientos, funciones y packages pueden encapsular operaciones repetitivas.

11.3 Seguridad

La evolución del sistema debe considerar privilegios, permisos de ejecución y acceso a tablas, aplicando el principio de otorgar solamente los permisos necesarios.

11.4 Control de versiones

Se recomienda utilizar Git y GitHub para versionar DDL y PL/SQL, manteniendo una rama principal, una rama de desarrollo y ramas de trabajo para nuevas funcionalidades o correcciones. Los commits pequeños y descriptivos facilitan identificar los cambios realizados.

11.5 Limitaciones

El procesamiento fila a fila puede aumentar su costo con grandes volúmenes. También debe vigilarse el tamaño máximo de colecciones como VARRAY y la complejidad de la lógica procedural a medida que el sistema crezca.

12. Conclusiones

12.1 Cumplimiento e impacto

El proyecto permite disponer de una base de datos Oracle orientada a la gestión de un hotel y sus reservas. La estructura relacional centraliza la información y establece relaciones entre huéspedes, habitaciones, reservas, estadías, servicios, cuentas y pagos.

La primera evaluación demuestra cómo PL/SQL puede complementar la estructura relacional mediante bloques anónimos, tipos compuestos, cursores, loops y excepciones. El caso de disponibilidad representa el vínculo más claro entre una necesidad del negocio y una solución procedural.

12.2 Aprendizajes

RECORD permite agrupar campos relacionados; VARRAY permite manejar colecciones acotadas; los cursores permiten recorrer conjuntos de resultados; los loops permiten aplicar procesamiento repetitivo y las excepciones permiten controlar situaciones previstas y reglas específicas.

12.3 Mejoras futuras

Para la segunda evaluación se recomienda continuar con procedimientos, funciones, packages y triggers. Posteriormente podría integrarse la base de datos con un backend Java utilizando Spring Boot y servicios REST, además de una interfaz web con HTML, CSS y JavaScript y una aplicación móvil en Kotlin. Estas integraciones corresponden a una evolución futura.

12.4 Conclusión académica y personal

El desarrollo permitió aplicar los contenidos de modelamiento relacional, integridad de datos y PL/SQL sobre un caso de negocio concreto. El proyecto muestra que cada recurso debe seleccionarse según el problema: una restricción, una consulta, un RECORD, un cursor o una excepción cumplen funciones diferentes.

Como experiencia de aprendizaje, el trabajo permitió avanzar desde el almacenamiento de datos hacia su procesamiento y control, comprendiendo mejor el rol de la base de datos dentro de una solución informática.

13. Anexos

Anexo A — Modelo de datos

Los modelos lógico y relacional se encuentran incorporados en la sección 4.1 para mantenerlos vinculados directamente con la explicación del diseño.

Anexo B — Script DDL y PL/SQL

La entrega complementaria contiene el script de creación de tablas, restricciones, datos iniciales, secuencias y ejercicios PL/SQL desarrollados para el proyecto.